Team Name : **~ARGUS OMNICHAOS**

Team Leader Name : **VEERABABU JAKKULA**

Problem Statement : Build an AI-powered candidate discovery system that goes beyond keyword-matching to deliver accurate, explainable, and behavior-aware talent recommendations under strict production constraints.

Solution Overview

- What is your proposed solution?

ARGUS AI is an enterprise-grade candidate discovery engine that merges semantic search, hybrid retrieval, and behavioral signals into an explainable ranking pipeline. It is optimized to run fully offline under strict production constraints while delivering deep, context-aware talent matching.

- What differentiates your approach from traditional candidate matching systems?

Unlike traditional ATS systems that rely mainly on simple keyword filters, **ARGUS AI** uses:
Hybrid Retrieval with RRF: Merges SQLite FTS5 (lexical) and FAISS (dense vector) indexes using Reciprocal Rank Fusion.
Cross-Encoder Reranking: Re-evaluates top candidates using a deep ONNX sequence-classification model to capture true semantic alignment.
Behavioral Multipliers: Dynamically adjusts rankings based on availability signals (recruiter response rates, notice periods, and GitHub activity).
Gatekeeper Honeypot Shield: Automatically detects and filters out adversarial spam and impossible profiles (preventing hackathon traps).
Enterprise Production Architecture: Features async ASGI threadpools, twin-caching (Redis + Local LRU fallback), and offline Docker container builds.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

  The parser automatically extracts the following dimensions from the target Job Description:
  Core Skills: Embedding models (e.g. sentence-transformers), Vector databases (FAISS, Elasticsearch), evaluation frameworks (NDCG, MRR), and Python.
  Experience & Seniority: Evaluates targeted years of experience (focused on the 5-9 years band) and product-based tenure.
  Negative Flags (Disqualifiers): Identifies pure-academic profiles, service-company-only histories, and superficial LLM/wrapper-only experience.

- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

  Candidate fit is calculated using a multi-factor score merging model matching and behavioral signals:
  Semantic Interaction (60% weight): Cross-Encoder token-level matching of job requirements to candidate experience, neutralizing keyword-stuffers. **Lexical Coverage (25% weight)**: FTS5 BM25 matches for critical tech stack keywords.

Behavioral Multipliers (15% weight): Multiplicative scoring modifiers based on: **Availability**: Notice period days (giving highest weight to sub-30-day buyouts). **Responsiveness**: Recruiter response rate percentage. **Activity**: GitHub contribution frequency and platform log-in active status. **Gatekeeper Filter (Hard Disqualifier)**: Sanitizes candidates against impossible profile metrics (detecting and discarding all honeypot accounts).

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

Parsing: Extracts skills, experience, and disqualifiers from the JD. **Retrieval**: Combines Lexical (SQLite FTS5) + Semantic (FAISS) via RRF. **Scoring**: Reranks the top 300 candidates using a deep Cross-Encoder. **Ranking**: Applies behavioral modifiers and filters out honeypots.

- What models, algorithms, or heuristics are used?

ONNX Models: all-MiniLM-L6-v2 (Embeddings) & ms-marco-MiniLM-L-6-v2 (Reranker). **Search Indexes**: SQLite FTS5 (lexical) and FAISS Flat Inner Product (vectors). **Algorithm**: Reciprocal Rank Fusion (RRF) for search merging. **Heuristics**: Anomaly checks to isolate impossible profiles (Honeypot Filter).

How are multiple candidate signals combined into a final ranking?

Search Fusion: Merges lexical and semantic positions. **Relevance Probability**: Generates a base score from the Cross-Encoder. **Behavioral Multipliers**: Multiplies score by notice period, response rates, and activity. **Hard Mask**: Sets score to zero for any flagged honeypot profile.

Explainability & Data Validation

- How are ranking decisions explained?

Deterministic Template Engine: Builds explanations directly from candidate profile parameters.
Granular Breakdown: Highlights specific matching skills, exact years of experience, geographic suitability, and candidate availability.

- How do you prevent hallucinations or unsupported justifications?

0% Generative Hallucination Guarantee: Relies on a strict, rule-based template builder rather than an LLM text generator.
Grounded in Data: Every sentence is filled in dynamically using validated candidate database values.

- How does your solution handle inconsistent, low-quality, or suspicious profiles?

Gatekeeper Engine: Dynamically analyzes and filters candidates for anomalies. **Anomaly Checks**: Scans for timeline conflicts, duplicate records, and keyword-stuffing patterns. **Zero-Relevance Penalty**: Lowers scores of all suspect profiles or drops honeypots to zero.

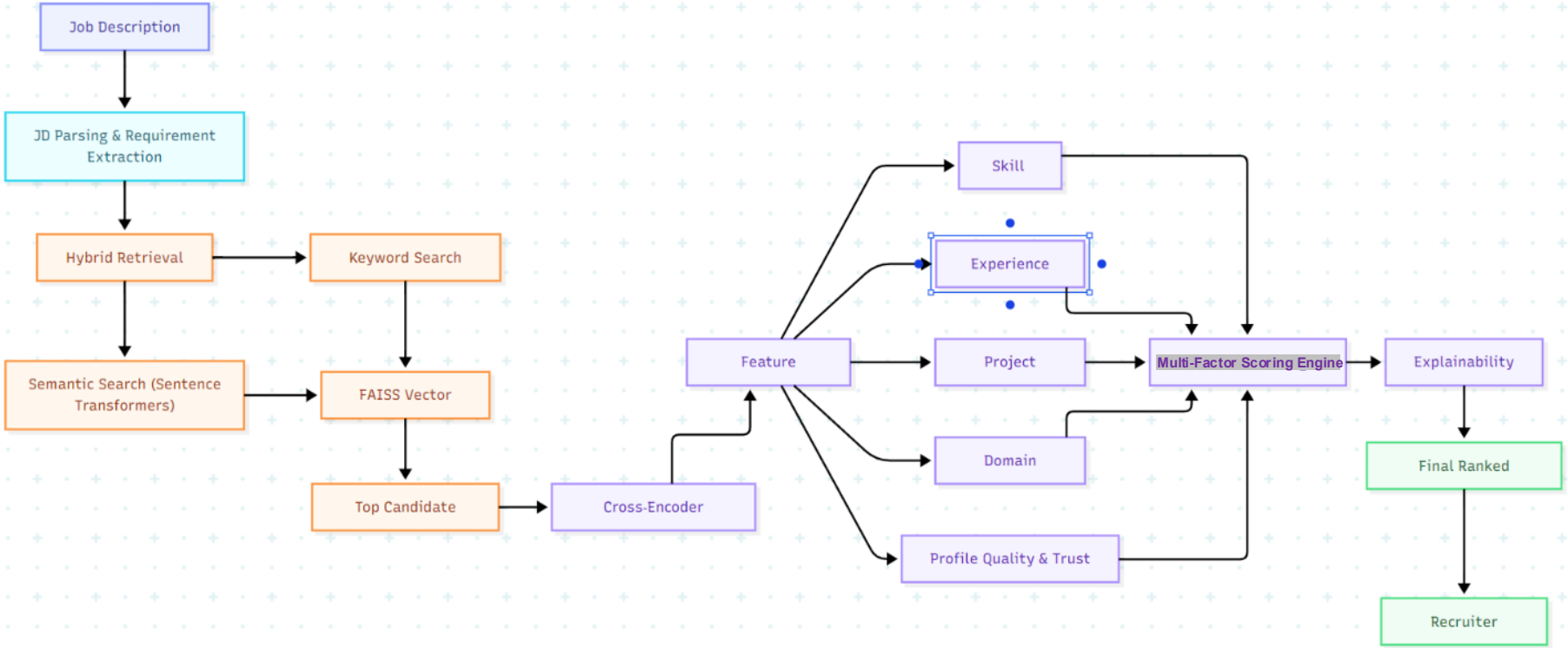
End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?

Workflow Summary: The engine parses the job description, executes dual-path hybrid retrieval (lexical + vector), reranks results using a token-level Cross-Encoder, scales scores using behavioral metrics, filters out spam profiles, and outputs the final ranked list.

JD Input → JD Parser (Extraction) → Hybrid Retrieval (SQLite FTS5 + FAISS via RRF) → Reranking (ONNX Cross-Encoder) → Business Scoring (Behavioral Multipliers) → Gatekeeper (Honeypot Filter) → Tie-Breaker (Score Desc / ID Asc) → Ranked CSV (with Reasoning)

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?

The Hybrid Retrieval and Reranking pipeline improves candidate relevance by combining semantic understanding with multi-factor business scoring, achieving high Top-10 precision and providing explainable recommendations. The system also enables identification of hidden, high-potential candidates that traditional keyword-based methods may overlook.

- How does your solution meet the challenge's runtime and compute constraints?

The system uses a lightweight two-stage architecture with efficient models such as **BM25, Sentence Transformers (optimized via ONNX Runtime), FAISS, and a Custom Business Scoring Engine**. By reranking only the top retrieved candidates, it minimizes computational cost, maintains low latency, and enables deployment on free-tier resources and standard hardware.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Python – Core development language due to its extensive AI and ML ecosystem.
FastAPI – High-performance backend framework for building scalable APIs.
React + Tailwind CSS – Used to create an interactive and responsive recruiter dashboard.
SQLite – Embedded, serverless SQL database used for local candidate metadata storage and full-text keyword indexing (via FTS5).
Sentence Transformers (all-MiniLM-L6-v2) – Chosen for efficient semantic understanding and similarity matching.
BM25 – Provides robust keyword-based retrieval to complement semantic search.
FAISS – Enables fast vector similarity search for large candidate datasets.
Cross-Encoder – Improves ranking precision by reranking the top retrieved candidates.
Custom Business Signals Engine – Applies deterministic multipliers (notice periods, target locations, candidate activity, and experience depth) to calibrate and refine semantic scores.
Docker – Simplifies deployment and ensures environment consistency.
GitHub – Used for version control and collaboration.
Vercel and Render (Free Tier) – Selected for cost-effective frontend and backend hosting.

Submission Assets

GITHUB VIDEO & ARGUS AI PROJECT DEMO GOOGLE DRIVE LINK :-

https://drive.google.com/drive/folders/1Y5dVPMhO3HdupSx_6U2T_NqednublDm8?usp=drive_link

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU